

PROGRAMACIÓN II

Guía de ejercicios: TDA Pila, TDA Cola y TDA Cola con Prioridad

Indicaciones generales

Justificá las respuestas cuando se solicite.

En los ejercicios de programación, utilizá únicamente las operaciones públicas del TDA indicado.

No está permitido acceder directamente a arreglos, nodos, índices, contadores u otros detalles internos de implementación, salvo que el ejercicio lo indique expresamente.

Cuando se indique que una estructura debe quedar intacta, debe conservar exactamente los mismos elementos y el mismo orden al finalizar el método.

Convención para Cola con Prioridad en esta guía: P=0 es la prioridad más alta; valores mayores representan prioridades menores.

Operaciones disponibles

TDA	Operaciones
Pila	InicializarPila, Apilar, Desapilar, Tope, PilaVacía
Cola	InicializarCola, Acolar, Desacolar, Primero, ColaVacía
Cola con Prioridad	InicializarCola, AcolarPrioridad, Desacolar, Primero, Prioridad, ColaVacía

EJERCICIO 1 - Seguimiento de una pila (8 puntos)

Partiendo de una pila vacía P, ejecutá las siguientes operaciones:

```
P.InicializarPila();
P.Apilar(8);
P.Apilar(3);
P.Apilar(12);
P.Desapilar();
P.Apilar(7);
P.Apilar(5);
System.out.println(P.Tope());      // (a)
P.Desapilar();
P.Desapilar();
System.out.println(P.Tope());      // (b)
P.Apilar(15);
System.out.println(P.Tope());      // (c)
```

- Indicá los tres valores mostrados por pantalla.
- Escribí el estado final de la pila en formato: base -> ... -> tope.
- ¿Qué elemento fue el primero en ingresar a la pila?
- ¿Qué elemento sería el próximo en salir?

EJERCICIO 2 - Contar elementos sin modificar la pila (10 puntos)

Implementá el siguiente método externo:

```
static int cantidadElementos(PilaTDA p)
```

Debe devolver la cantidad de elementos almacenados en p. La pila debe quedar exactamente igual al finalizar. Usá solamente operaciones del TDA Pila.

Ejemplo: base -> 4 -> 8 -> 2 -> 6 -> tope => retorna 4 y la pila queda sin cambios.

EJERCICIO 3 - Buscar un elemento sin modificar la pila (10 puntos)

Implementá:

```
static boolean contiene(PilaTDA p, int x)
```

Debe retornar true si x aparece en la pila y false en caso contrario. La pila debe quedar intacta al finalizar. Usá solamente operaciones del TDA Pila.

Además, explicá brevemente por qué necesitás una estructura auxiliar para resolverlo.

EJERCICIO 4 - Seguimiento de una cola (8 puntos)

Partiendo de una cola vacía C:

```
C.InicializarCola();
C.Acolar(5);
C.Acolar(9);
C.Acolar(2);
C.Acolar(8);
System.out.println(C.Primer());    // (a)
C.Desacolar();
C.Desacolar();
C.Acolar(6);
C.Acolar(11);
System.out.println(C.Primer());    // (b)
C.Desacolar();
System.out.println(C.Primer());    // (c)
```

- Indicá los tres valores mostrados.
- Escribí el estado final: Primero -> ... -> último.
- ¿Qué elemento fue el último en ingresar?
- ¿Cuál sería el próximo elemento en salir?

EJERCICIO 5 - Sumar los elementos de una cola conservando el orden (10 puntos)

Implementá:

```
static int sumar(ColaTDA c)
```

Debe retornar la suma de todos los elementos. La cola debe conservar todos sus elementos y el mismo orden original.

Ejemplo: Primero -> 3 -> 8 -> 4 -> 10 -> último => retorna 25.

EJERCICIO 6 - Eliminar todas las apariciones de un valor (12 puntos)

Implementá:

```
static void eliminar(ColaTDA c, int x)
```

Debe eliminar de la cola todas las apariciones de x, preservando el orden relativo del resto de los elementos. Usá solamente operaciones del TDA Cola.

Ejemplo: Primero -> 8 -> 3 -> 5 -> 3 -> 9 -> 3 -> último; al ejecutar eliminar(c,3), debe quedar Primero -> 8 -> 5 -> 9 -> último.

EJERCICIO 7 - Seguimiento de una Cola con Prioridad (10 puntos)

Se trabaja con tres prioridades: P=0 alta, P=1 media y P=2 baja. Partiendo de una Cola con Prioridad vacía CP:

```
CP.AcolarPrioridad(20, 2);
CP.AcolarPrioridad(15, 0);
CP.AcolarPrioridad(8, 1);
CP.AcolarPrioridad(12, 0);
CP.AcolarPrioridad(30, 2);
CP.AcolarPrioridad(6, 1);
System.out.println(CP.Primer()); // (a)
System.out.println(CP.Prioridad()); // (b)
CP.Desacolar();
CP.Desacolar();
System.out.println(CP.Primer()); // (c)
System.out.println(CP.Prioridad()); // (d)
```

Respondé:

Los valores de (a), (b), (c) y (d).

El estado de las tres subcolas antes de los dos Desacolar.

El estado de las tres subcolas después de los dos Desacolar.

EJERCICIO 8 - Contar elementos de una prioridad (12 puntos)

Implementá:

```
static int contarPrioridad(ColaPrioridadTDA cp, int p)
```

Debe devolver cuántos elementos almacenados tienen prioridad p. La Cola con Prioridad debe quedar exactamente igual al finalizar. Usá solamente las operaciones públicas del TDA.

EJERCICIO 9 - Eliminar una prioridad completa (12 puntos)

Implementá:

```
static void eliminarPrioridad(ColaPrioridadTDA cp, int prioridad)
```

Debe eliminar todos los elementos que tengan la prioridad indicada y conservar el resto con sus prioridades y orden relativo original.

Ejemplo inicial:

```
Subcola P=0: [8, 3]  
Subcola P=1: [7, 5, 4]  
Subcola P=2: [10, 12]
```

Luego de eliminarPrioridad(cp,1), debe quedar:

```
Subcola P=0: [8, 3]  
Subcola P=1: vacía  
Subcola P=2: [10, 12]
```

EJERCICIO 10 - Seleccionar el TDA adecuado (8 puntos)

Para cada situación, indicá si corresponde utilizar Pila, Cola o Cola con Prioridad y justificá brevemente la elección.

1. Sistema que mantiene las últimas acciones realizadas para implementar Deshacer.
2. Turnos atendidos estrictamente por orden de llegada.
3. Procesamiento de incidentes donde los críticos deben atenderse antes que los normales.
4. Historial de páginas para implementar el botón Atrás.
5. Trabajos enviados a una impresora y atendidos por orden de llegada.
6. Solicitudes de soporte clasificadas como crítica, alta, media y baja.
